

15 Proposed Final Assessment Projects

1. Public Services Chatbot for Citizen Query Resolution

PROBLEM STATEMENT

Public service agencies receive a high volume of citizen inquiries ranging from service eligibility to application procedures. Currently, many queries are handled manually via call centers or email, causing long wait times and inconsistent responses. This leads to citizen frustration, increased workload on staff, and delays in service delivery. Existing FAQ pages are static and insufficient for personalized assistance. Automating initial query resolution with a conversational AI agent can streamline service, provide 24/7 support, and free human agents for complex cases, enhancing citizen satisfaction and operational efficiency.

Data Considerations:

Key data includes structured FAQs and policy documents from public service websites and internal knowledge bases in text and PDF formats. Preprocessing involves text extraction, BPE subword tokenization (tiktoken), paragraph chunking, and dense vector embedding generation using text-embedding-3-small. Synthetic query-response pairs can be generated from existing FAQs for training, testing, and evaluation. Care should be taken with sensitive information, ensuring strict data privacy and relevance to the problem context.

SOLUTION EXPECTATIONS

A prototype conversational AI agent exposing a layered FastAPI backend (routers/, services/, schemas/) protected by header-based authentication (Depends). The AI core utilizes gpt-4.1-mini integrated into a LangGraph StateGraph workflow featuring retrieve, grade_context, rewrite_question (with retry loops up to max_attempts), generate_answer (with strict inline source citations), and fallback_answer nodes to prevent hallucinations when documents are missing. The retrieval and query grading flow is additionally packaged as a Model Context Protocol (MCP) server so external AI clients can invoke the citizen search capability dynamically.

2. Generative AI Assistant for Customized Manufacturing Training Content

PROBLEM STATEMENT

Manufacturing plants face challenges in delivering tailored training content that reflects specific machinery, processes, and safety protocols relevant to different operator roles. Traditional training materials are generic, static, and often outdated, resulting in lower engagement and

ineffective knowledge transfer. Updating training content manually is resource-intensive and slow. There is a need for an AI assistant that dynamically generates customized training modules, calculates required instruction hours, and creates quizzes based on latest process data and user skill levels.

Data Considerations:

Existing training manuals, equipment operating guidelines, and safety protocols in text and markdown formats. User profiles, skill assessments, and training history accessible via mock APIs. Preprocessing involves paragraph chunking, skill tagging, and embedding indexing. Synthetic data can simulate learner profiles and training scenarios for testing, ensuring high data quality and relevance.

SOLUTION EXPECTATIONS

A web-based interactive service built on a layered FastAPI architecture using Pydantic models (BaseModel, Field) for strict request validation and schema enforcement. Driven by gpt-4.1-mini, the backend leverages LangChain @tool definitions (get_operator_skill_profile, calculate_training_hours, search_safety_manuals) bound to the model via bind_tools. An iterative tool-calling loop handles HumanMessage, AIMessage, and ToolMessage execution to produce tailored training summaries and quiz modules. The manual search and training calculation tools are exposed as an MCP server for integration into plant LMS interfaces.

3. Business Analysis Automated Impact Analysis Summarizer

PROBLEM STATEMENT

Business analysts often perform impact analysis to understand how proposed changes affect systems, processes, and stakeholders. This requires consolidating information from multiple documents and stakeholder inputs, which is time-consuming and prone to omissions. Existing tools do not automate impact summarization or provide consistent formats, leading to delays and inconsistent documentation quality. An AI summarizer that generates clear, structured impact analysis reports from input documents can enhance analyst productivity and project alignment.

Data Considerations:

Change request documents, stakeholder feedback notes, and system architecture specifications in text or PDF format. Preprocessing includes text extraction, chunking, and dense vector index generation. Synthetic change requests can be used for development and testing while maintaining strict data quality and domain relevance.

SOLUTION EXPECTATIONS

An AI summarization tool built using a layered FastAPI framework with dedicated Pydantic input

and output schemas. The solution employs dense vector embeddings (text-embedding-3-small) and cosine similarity search to retrieve relevant specification passages. An LCEL pipeline (ChatPromptTemplate | ChatOpenAI | StrOutputParser / structured output parser) orchestrates gpt-4.1-mini with low temperature settings to synthesize multi-document inputs into standardized JSON impact reports containing fields for affected systems, risk levels, and mitigation steps. The impact analysis pipeline is packaged as an MCP resource provider for seamless integration with business analysis tooling.

4. AI Chatbot for Insurance Policy Purchase Guidance

PROBLEM STATEMENT

Insurance customers often find it difficult to select suitable policies due to complex products and unclear benefits. Customer service agents are overloaded with queries, leading to delays and inconsistent advice. Existing websites provide static product descriptions without interactive support tailored to individual needs. There is a need for a conversational AI assistant that can guide customers through policy options, calculate custom premium quotes, answer questions in natural language, and provide personalized recommendations based on customer profiles.

Data Considerations:

Insurance product catalogs containing policy features, premiums, and coverage options in structured JSON formats, alongside policy rule PDFs. Preprocessing includes data normalization, vector embedding indexing, and CRM attribute mapping. Synthetic customer queries and profiles are recommended for development while maintaining privacy safeguards.

SOLUTION EXPECTATIONS

An AI chatbot backend implemented in FastAPI with modular routers, services, and Pydantic schemas, protected by token-based authentication dependencies. The solution uses gpt-4.1-mini inside a LangGraph routing engine: queries are classified and directed via conditional edges either to a RAG sub-graph (calculating cosine similarity over policy PDF chunks) or to a tool-calling sub-graph that invokes custom functions like `calculate_premium(coverage_amount, deductible)`. The policy search and quote calculation capabilities are packaged into an MCP server to enable third-party aggregator agents to consult the guidance engine.

5. Conversational Inventory Audit Assistant for Consumer Goods Distribution

PROBLEM STATEMENT

Distribution centers for consumer goods regularly perform inventory audits to reconcile stock levels and identify discrepancies. Manual audit processes are labor-intensive, error-prone, and slow, delaying corrective actions. Auditors and managers lack an interactive tool to query inventory data, track audit progress, and generate discrepancy reports efficiently. Existing

systems do not leverage conversational AI to facilitate natural language queries and real-time updates, increasing shrinkage risks and operational costs.

Data Considerations:

Inventory database records including stock counts, SKU details, location codes, and status logs formatted as JSON or relational tables. Preprocessing involves data validation, normalization, and state synchronization. Synthetic data can simulate warehouse audit scenarios with typical discrepancies to test edge cases safely.

SOLUTION EXPECTATIONS

An interactive conversational backend built with FastAPI layer architecture (routers/inventory.py, services/audit.py, schemas/inventory.py). Powered by gpt-4.1-mini, the application integrates LangChain tools using the @tool decorator (query_stock_level, log_discrepancy, generate_audit_summary). A generic LangChain tool-execution loop manages multi-step tool calls, argument validation error handling, and ToolMessage feedback to allow managers to record audit findings and generate discrepancy summaries via natural language. The database interaction tools are exposed as an MCP server for mobile handheld terminals.

6. Application Development Knowledge Assistant for IT Teams

PROBLEM STATEMENT

Application development teams frequently face challenges accessing up-to-date technical documentation, coding standards, and troubleshooting guides, leading to delays and inconsistent code quality. Developers spend significant time searching through fragmented resources and legacy documentation repositories. Existing knowledge management systems lack conversational interfaces and contextual understanding. There is a need for an AI-powered knowledge assistant that provides developers with instant, relevant answers to technical queries, code snippet recommendations, and best practices.

Data Considerations:

Internal technical documentation in Markdown, HTML, or PDF formats, alongside code snippet examples and issue-tracking logs. Preprocessing involves text extraction, subword tokenization with tiktoken, chunking, and embedding indexing using text-embedding-3-small. Synthetic logs and anonymized documentation excerpts can be used for development.

SOLUTION EXPECTATIONS

An AI-powered technical assistant backend built on FastAPI using layered separation of concerns. The AI core utilizes gpt-4.1-mini embedded within a LangGraph agentic RAG workflow. The graph implements a grade_context node that measures cosine similarity and keyword overlap; if context quality is weak (<0.25 similarity threshold), it enters a

rewrite_question loop before calling generate_answer with mandatory documentation citations (e.g., [doc_id]). The knowledge assistant and code retrieval capabilities are packaged as an MCP server for direct access within developer IDEs.

7. AI-Based Supplier Risk Assessment Report Generator for Capital Markets

PROBLEM STATEMENT

Capital markets firms rely on third-party suppliers for critical services but face challenges assessing and monitoring supplier risk due to fragmented data sources including financial reports, news, and regulatory filings. Manual risk assessment is time-consuming and lacks comprehensive insights, exposing firms to compliance, operational, and reputational risks. Risk managers struggle to aggregate and interpret diverse information rapidly to generate synthesized risk narratives, red-flag alerts, and actionable risk scores.

Data Considerations:

Supplier financial reports, news articles, and regulatory filings in text or PDF formats. Preprocessing requires OCR/text extraction, normalization, and vector embedding indexing. Anonymized or synthetic supplier profiles are used for development to ensure client data privacy.

SOLUTION EXPECTATIONS

A web application backend developed in FastAPI with structured Pydantic input/output schemas and custom error handlers. The solution uses text-embedding-3-small dense embeddings and numpy cosine similarity to retrieve relevant financial chunks. An LCEL pipeline combines ChatPromptTemplate and ChatOpenAI (gpt-4.1-mini, temperature=0.0) with structured output enforcement (.with_structured_output() / PydanticOutputParser) to generate structured JSON risk reports containing overall risk scores, identified red flags, financial health summaries, and compliance recommendations. The report generator is packaged as an MCP tool for enterprise risk dashboards.

8. Application Development Code Review Assistant

PROBLEM STATEMENT

Software development teams spend significant time performing manual code reviews to identify bugs, security issues, and style inconsistencies. This process can delay release cycles and lead to inconsistent quality due to reviewer subjectivity and workload variability. Automated static code analysis tools often produce many false positives and lack contextual understanding of code intent. Developers need an AI assistant that can analyze code snippets, provide concise feedback, suggest concrete fixes, and enforce coding standards.

Data Considerations:

Input data includes Python code snippets in plain text or markdown, public code review datasets, and synthetic code examples highlighting security vulnerabilities, syntax bugs, and style violations. Preprocessing involves syntax validation and AST checking.

SOLUTION EXPECTATIONS

An AI assistant service engineered with FastAPI layered architecture. Utilizing gpt-4.1-mini with system instructions defining a "Senior Security & Code Architect" persona, the application integrates LangChain tools (run_syntax_checker, lookup_style_guidelines). The agent runs an execution loop where code snippets are analyzed, syntax validation tools are called dynamically, and structured inline review comments detailing bug risks and style violations are returned. The code review engine is packed as an MCP server to integrate with automated git pull request workflows.

9. HR Policy & Employee Onboarding Compliance Assistant

PROBLEM STATEMENT

New employees struggle to navigate extensive company HR policies, leave rules, travel reimbursement limits, and remote work guidelines, leading to repetitive inquiries and administrative burden on HR teams. HR departments require an automated onboarding assistant capable of answering policy questions accurately, citing specific policy sections, and adhering strictly to documented rules while gracefully handling queries regarding missing or unauthorized topics.

Data Considerations:

Company policy documents (vacation policy, laptop replacement, travel reimbursement, security, and remote work) in text or PDF format. Preprocessing involves paragraph-based chunking and vector embedding generation using text-embedding-3-small. Anonymized or synthetic employee profiles are used for authorization testing.

SOLUTION EXPECTATIONS

A FastAPI backend structured into routers/, services/, and schemas/ layers, with endpoint access controlled by user role authentication dependencies (require_auth). The system implements a full LangGraph RAG StateGraph incorporating initialize, retrieve, grade_context, rewrite_question (retrying up to max_attempts), generate_answer (with enforced source citations), and fallback_answer nodes returning "I don't know based on the provided policy documents" when information is missing. The HR policy retrieval workflow is packaged as an MCP server for enterprise workspace chat platforms.

10. Customer Support Ticket Escalation Engine

PROBLEM STATEMENT

E-commerce customer support centers experience heavy request volumes for order tracking, refund eligibility, and product complaints. Manual processing causes delayed resolutions and inconsistent refund calculations. Support teams require an intelligent assistant capable of fetching real-time order statuses, automatically calculating refund amounts based on order state, and generating formal support escalation tickets for complex cases requiring human follow-up.

Data Considerations:

Mock customer order database records (order ID, shipping status, ETA, total price) and support ticket registries in JSON format. Data validation ensures strict schema compliance and prevents unauthorized record modifications.

SOLUTION EXPECTATIONS

A layered FastAPI backend separating route handling (routers/support.py), business logic (services/support.py), and validation schemas (schemas/support.py). Driven by gpt-4.1-mini, the backend defines LangChain tools via `@tool: get_order_status`, `calculate_refund_amount`, and `create_support_ticket`. A reusable tool-calling conversation loop processes user queries, executes requested tools sequentially or in parallel, handles invalid arguments gracefully, and returns grounded final responses. The support actions and ticket creation tools are packed into an MCP server for CRM system integration.

11. Corporate Financial Statement Analyzer

PROBLEM STATEMENT

Financial analysts spend considerable time manually searching through long quarterly earnings reports and PDF financial filings to extract key performance metrics such as revenue growth, EBITDA margins, and debt ratios. This manual extraction process is slow and susceptible to human error. An AI tool is needed to process multi-document financial filings, retrieve metric-specific passages, enforce citation accuracy, and generate structured financial comparison summaries.

Data Considerations:

Corporate quarterly earnings reports and SEC filing excerpts in PDF or text format. Preprocessing involves text extraction, chunking, and dense vector indexing with text-embedding-3-small. Synthetic earnings statements are used for dev/test validation.

SOLUTION EXPECTATIONS

A FastAPI service built using clean architecture layers and Pydantic response models. The application uses numpy and cosine similarity over dense vector embeddings to locate relevant financial passages. An LCEL chain (ChatPromptTemplate | ChatOpenAI | StrOutputParser) configures gpt-4.1-mini (temperature=0.0) with system instructions mandating strict inline citations (e.g., [Q3_Statement_Page_4]) and structured JSON outputs containing extracted metrics, quarter-over-quarter changes, and confidence scores. The financial retrieval and metric extraction engine is packaged as an MCP resource server.

12. Healthcare Appointment Triage & Clinic FAQ Assistant

PROBLEM STATEMENT

Outpatient clinics handle numerous phone and email queries regarding operating hours, pre-procedure preparation guidelines, and appointment scheduling. Clinic staff spend significant time providing repetitive information instead of focusing on patient care. A healthcare triage assistant is required to answer operational FAQs using grounded clinic documentation while routing scheduling requests to dynamic appointment booking tools under strict safety guardrails.

Data Considerations:

Clinic operational FAQs, pre-visit preparation documents, and doctor schedule datasets formatted as JSON. Medical diagnostic queries are explicitly filtered out using system prompt constraints. Synthetic patient profiles are utilized for testing.

SOLUTION EXPECTATIONS

A FastAPI layered application using Pydantic models to validate incoming patient payloads. Powered by gpt-4.1-mini, the system employs a LangGraph StateGraph with conditional routing: medical/operational FAQ questions route to a RAG pipeline utilizing text-embedding-3-small and cosine similarity retrieval, while scheduling requests route to tool nodes (check_slot_availability, book_appointment). Guardrail prompts ensure the model declines medical advice and directs emergencies to standard care. Clinic FAQ and booking tools are packaged into an MCP server.

13. Real Estate Property Finder & Mortgage Calculator

PROBLEM STATEMENT

Homebuyers struggle to find properties that match their lifestyle preferences and financial constraints because traditional search engines only filter by static tags and fail to provide personalized monthly cost estimates. A conversational real estate assistant is needed to parse natural language requirements, search property listings semantically, and calculate estimated monthly mortgage payments based on interest rates, down payments, and loan terms.

Data Considerations:

Property catalog listings (price, location, bedroom count, amenities description) in JSON format and current mortgage rate reference tables. Synthetic property datasets and buyer profiles simulate search scenarios.

SOLUTION EXPECTATIONS

A FastAPI service organized into routers/properties.py, services/finance.py, and schemas/properties.py. Utilizing gpt-4.1-mini, the application binds custom LangChain tools (search_property_catalog, calculate_mortgage_payment). A LangChain tool-execution loop manages multi-step interactions—first searching the property catalog semantically, then invoking the mortgage calculator with extracted loan parameters, and returning a comprehensive recommendation summary. The property search and mortgage calculation tools are exposed as an MCP server.

14. Enterprise IT Desk Incident Resolver

PROBLEM STATEMENT

Enterprise IT helpdesks face large volumes of repetitive incident tickets for issues like VPN connectivity drops, password resets, and software license requests. IT support engineers need an automated assistant that can retrieve technical knowledge base resolution steps, evaluate solution confidence, and automatically update ticket resolution status in helpdesk systems.

Data Considerations:

IT Knowledge Base resolution articles in Markdown and synthetic incident ticket logs (ticket ID, issue description, error codes, priority) formatted as JSON. Preprocessing includes paragraph chunking and vector indexing using text-embedding-3-small.

SOLUTION EXPECTATIONS

A layered FastAPI backend with header-based authorization dependencies securing administrative operations. The system uses gpt-4.1-mini inside a LangGraph agentic workflow: the graph retrieves KB articles via cosine similarity, executes a grade_context node to assess error-code match confidence, rewrites ambiguous queries if confidence is low, and invokes an update_ticket_status tool node when a confident fix is generated. The IT incident resolution workflow is packed as an MCP server for integration into ITSM platforms (e.g., ServiceNow, Jira).

15. E-Commerce Personal Shopping & Comparison Assistant

PROBLEM STATEMENT

Online shoppers find it difficult to compare complex technical specifications across multiple products (such as laptops or smartwatches) using standard product pages. They require a

personal shopping assistant capable of searching product catalogs using natural language preferences, extracting technical specifications, comparing items side-by-side, and generating structured comparison tables highlighting pros, cons, and value ratings.

Data Considerations:

Product catalog descriptions, detailed technical specification sheets, and customer review summaries in text/JSON formats. Preprocessing involves BPE tokenization, chunking, and embedding generation using text-embedding-3-small.

SOLUTION EXPECTATIONS

A FastAPI layered application exposing Swagger-documented endpoints for product search and comparison. The system implements dense vector embeddings and cosine similarity search to retrieve candidate products matching natural language queries. An LCEL chain (prompt | llm | parser) orchestrates gpt-4.1-mini (temperature=0.5) to analyze retrieved specification documents and format structured Markdown comparison tables containing feature breakdowns, target user fit, and concise purchase recommendations. The product comparison and catalog search tools are packaged as an MCP server.